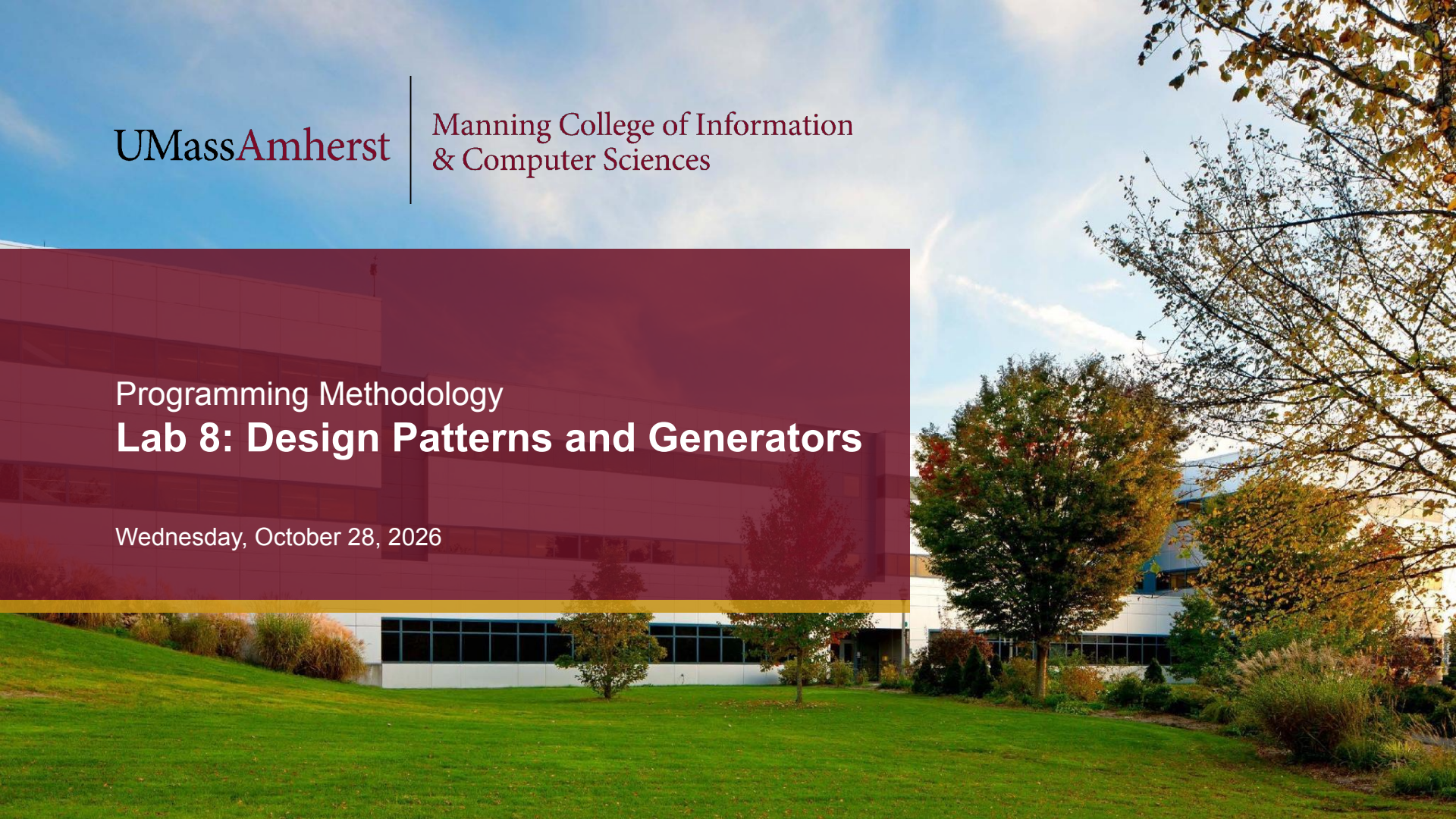


Programming Methodology

Lab 8: Design Patterns and Generators

Wednesday, October 28, 2026

Reminders

- **Midterm 2 is next Wednesday (11/4)**
  - **Time: 7-9 pm**
  - **Location: Mahar 108**

Reminders

- Homework 7a is due tonight (10/28) at 11:59 pm.
  - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
  - Fill out the excusal form on Canvas before the lab.
  - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
  - Lab submissions are due at 6:00 pm.
  - There are **two Gradescope assignments**: "Lab 8 Design Pattern Questions" and "Lab 8 Code". Both counts towards your lab grade.

Groups

If you aren't already, please sit with your new groups from last week!

The MVC exercise involves open-ended conceptual questions.

- You will be asked to talk to each other about them.

Singleton Review

The Singleton pattern ensures that a class has only one instance and provides a shared and controlled access point for that object.

Consider: When is shared state useful? What problems can it cause?

MVC Review

The key components of the **Model-View-Controller** design pattern are:

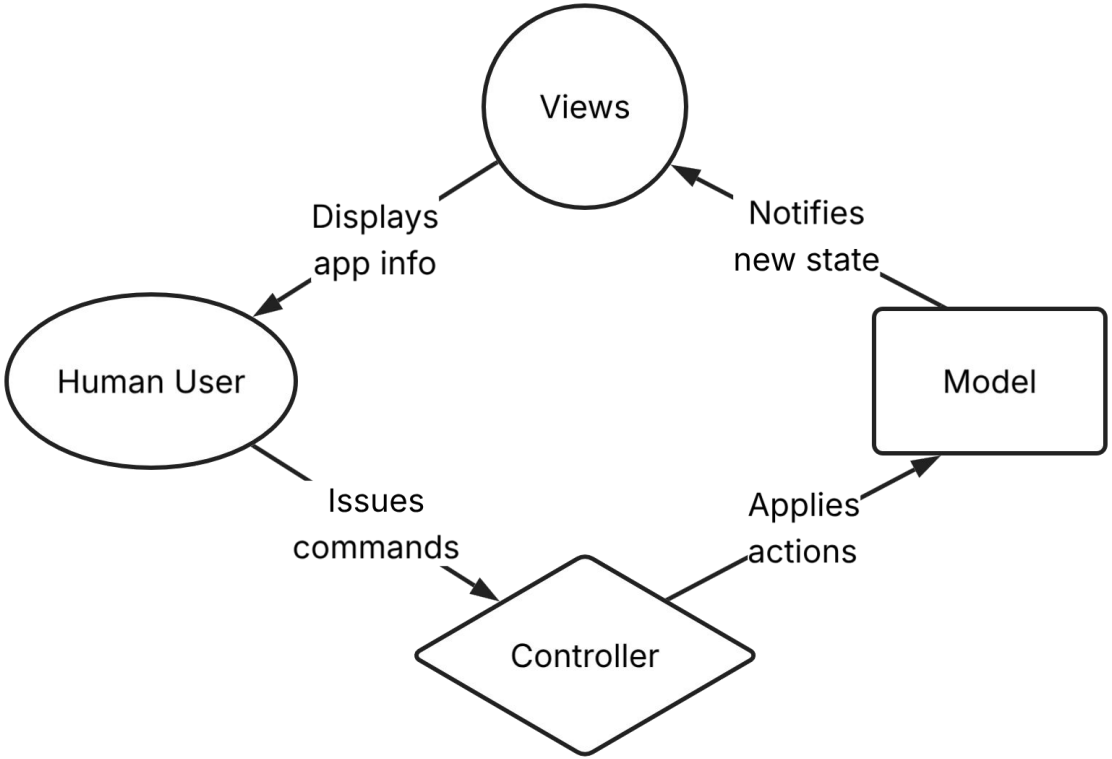
- **Model**: Encapsulates core data and logic of application
- **View**: Displays application state to the user
- **Controller**: Processes user commands and manages views

A key design principle of MVC:

- Application state and logic are *decoupled* from the user interface.

We won't review all of the details, because your first exercise ask you to talk about them!

MVC Review


Views
Displays
app info
Notifies
new state
Human User
Model
Issues
commands
Applies
actions
Controller

Exercise 1: Design Patterns

Discuss the following questions with your group and enter your answers individually into the Gradescope assignment titled "Lab 8 Design Pattern Questions".

1. What is a design pattern? Why might we use one?

2. What is the purpose of the Singleton pattern? What are its key characteristics?

3. Describe the nature of interactions between:
   - Model and View
   - Model and Controller
   - View and Controller

Solution 1: Design Patterns

1. What is a design pattern? Why might we use one?

   A design pattern is a well-known solution that acts like a blueprint for solving a specific type of problem.

   Consider using one when you recognize a common design problem!

Solution 1: Design Patterns

2. What is the purpose of the Singleton pattern? What are its key characteristics?

   The purpose of the Singleton pattern is to provide controlled access to a piece of global state.

   It guarantees that there will only be one instance of a class, makes that instance globally accessible, and provides controlled access to its state.

Solution 1: Design Patterns

3. Describe the nature of interactions between:

   - Model and View

     The Model notifies a View of state updates through a generalized interface, without depending on their concrete implementations.

     A View receives updates from the Model, but it does not access its state directly.

Solution 1: Design Patterns

3. Describe the nature of interactions between:

   - Model and Controller

     The Controller calls methods of the Model based on user action. This can include attaching or detaching View instances.

     The Model communicates state changes to Views, not directly to the Controller.

Solution 1: Design Patterns

3. Describe the nature of interactions between:

   - View and Controller

     The Controller can open and close Views, but all state changes go through the Model.

     The View displays information directly to the user rather than going through the Controller. Note that in some implementations, Views also handle user input and thus communicate that back to the Controller.

Generators Review

Generators allow us to produce values one at a time without manually tracking iterator state.

- `Generator` follows the `IterableIterator` interface.
- It's important to know how to create iterators and when to use them, but generators can be more concise and easier to work with.

Generators Review

Generators are created by **generator functions** that have the following syntax:

```
function* numbers(): Generator<number> {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numbers();

gen.next().value // 1
gen.next().value // 2
gen.next().value // 3
gen.next().value // undefined
```

`function*` defines a generator function.

`yield` produces a value and pauses execution.

`.next()` resumes execution until the next `yield`.

When the function completes, `.next()` will return `{ done: true, value: undefined }`

The generator function takes care of making the `IteratorResult`s for you!

Exercise 2: Generators

Implement a generator function `mergedGenerator` that takes in two number iterators, each yielding values in non-decreasing order, and merges them into a single non-decreasing sequence.

Your solution should:

- Preserve duplicates and return all values from both iterators.
- Handle empty iterators and cases where one iterator finishes before the other.
- Use O(1) space.

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  // ...
}
```

We need to keep track of the next value from each iterator.

- `r1` and `r2` store the current `IteratorResult`s.
- Each result contains `value` and `done`.
- We only advance an iterator after yielding its current value.

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  while (!r1.done || !r2.done) {

    // ...
  }
}
```

We use a `while` loop to continue the generator until both iterators are exhausted.

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  while (!r1.done || !r2.done) {
    if (r1.done) {
      yield r2.value;
      r2 = it2.next();
    }

    // ...
  }
}
```

If the first iterator is exhausted, we `yield` from the second one.

- The next time `.next()` is called, execution will resume and we will advance `r2`.

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  while (!r1.done || !r2.done) {
    if (r1.done) {
      yield r2.value;
      r2 = it2.next();
    } else if (r2.done) {
      yield r1.value;
      r1 = it1.next();
    }

    // ...
  }
}
```

Similarly, if the second iterator is exhausted, we `yield` from the first one (and advance at next call).

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  while (!r1.done || !r2.done) {
    if (r1.done) {
      yield r2.value;
      r2 = it2.next();
    } else if (r2.done) {
      yield r1.value;
      r1 = it1.next();
    } else if (r1.value <= r2.value) {
      yield r1.value;
      r1 = it1.next();
    }

    // ...
  }
}
```

If neither iterator is exhausted, we need to compare the current values.

If the first value is less than (or equal to) the second value, we `yield` the first value.

Solution 2: Generators

```
function* mergeGenerator(it1: Iterator<number>, it2: Iterator<number>): Generator<number> {
  let r1 = it1.next();
  let r2 = it2.next();

  while (!r1.done || !r2.done) {
    if (r1.done) {
      yield r2.value;
      r2 = it2.next();
    } else if (r2.done) {
      yield r1.value;
      r1 = it1.next();
    } else if (r1.value <= r2.value) {
      yield r1.value;
      r1 = it1.next();
    } else {
      yield r2.value;
      r2 = it2.next();
    }
  }
}
```

Otherwise, the second value is lesser, and we `yield` that one.

Solution 2: Generators

Key takeaways:

- Generator functions use the `function*` and `yield` syntax.
  - Remember, you can also `yield*` from an iterable!
- When using iterators within generator functions, you still need to call `.next()` on them and handle their `IteratorResults`.
  - But, you only yield a value, not an `IteratorResult`!
- A `for` or `while` loop is often necessary to facilitate repeated yielding.
- Generators can be easier to implement (and more readable) than iterators because you don't have to construct the `IteratorResult` directly.